Supporting Documentation

BUSI4373 — Churn Prediction for FoodCorp

1. How to Run the Notebook and Model

The submission consists of a single Databricks notebook (BUSI4373_20811152_2526) with mixed SQL and Python cells. The notebook is executed end to end on a Databricks cluster with the standard ML runtime; no additional library installations are required beyond what the first cell installs via pip.

Prerequisites

- Databricks workspace with read/write access to the workspace.ml34 catalog.
- Databricks ML runtime (any recent version supporting Python 3.10+, scikit-learn 1.1+ and PySpark 3.4+).
- The notebook expects the FoodCorp ml34 dataset to be already mounted at workspace.ml34. The first cell installs the urllib-fetched dataset loader provided by the module.

Execution sequence

The notebook is designed to be executed top to bottom. Cells follow the analytical pipeline as described in the report: (i) schema and integrity checks, (ii) creation of the flat view ml34.v_transactions, (iii) inter-visit gap analysis and β selection, (iv) β sensitivity, (v) feature-engineering function definitions, (vi) creation of feature/label tables for R = 300, 400, 500, (vii) data loading into pandas, (viii) preprocessing and scaling, (ix) baselines and default models, (x) meta-parameter tuning, (xi) permutation importance and feature selection check, (xii) final retraining on train+validation and one-shot test evaluation, (xiii) threshold analysis, (xiv) pen portraits and technical interpretation.

Reproducibility is supported by a fixed random_state = 42 on every stochastic estimator, a fixed RNG seed in permutation_importance, and a deterministic train/validation/test scheme using fixed reference days R = 300, 400, 500. Re-running the notebook on the same cluster reproduces every number reported in the report.

Adapting the model to a new reference date

FoodCorp's in-house team can re-score active customers at any future reference date by calling the two parameterised functions defined in the notebook:

```
save_features(R, beta=49, w_size=14, n_windows=4)
save_labels(R, beta=49)
```

These regenerate ml34.features_R{R} and ml34.labels_R{R} for any chosen R. The model object final_model (a fitted HistGradientBoostingClassifier or RandomForestClassifier depending on the best_name selected by the tuning step) can then be re-applied to the new feature table after loading via load_dataset(R) and applying the train-fitted scaler.

2. Additional Cleaning Performed

The raw FoodCorp tables required limited but specific cleaning beyond joining. The decisions taken and their justification are summarised below; full implementation is in the notebook cells named in section 3.

- Sentinel value 9,999,999 in receipt_lines.value: a small number of rows carry this obviously-non-genuine spend value. It is treated as missing in v_transactions through a derived value_clean column and is flagged by an is_value_outlier indicator so rows can still be tracked but excluded from spend aggregations.
- Negative line values: these are kept because they represent legitimate returns and refunds. They naturally lower window spend totals, which is consistent with the customer behaviour (real spend net of returns).

- Referential integrity: explicit LEFT-JOIN-based orphan checks (receipts→customers and receipts→stores) confirmed 0 orphans before constructing v_transactions.
- Age missingness: age_at_R is computed from customers.dob but is null for 86–88% of customers across all three splits. The column is dropped from the modelling feature set rather than imputed, because imputing nearly 9 out of 10 rows with a single median value would inject more noise than signal.
- Class imbalance: the 70/30 non-churn/churn split is handled by reporting both F1 and ROC-AUC and by exposing the operating-threshold trade-off, not by re-balancing the training data. class_weight = 'balanced' was tested during tuning but produced a slightly lower ROC-AUC and more variable predictions, so it was not adopted.

3. Figure / Table to Notebook Cell Mapping

Every figure and table referenced in the main report is generated by a specific cell or code block in the Databricks notebook. The mapping below names the figure/table, the saved filename (where applicable), the markdown section heading of the producing cell, and the principal Spark / scikit-learn / matplotlib operation that creates it.

Table A — Report assets and their notebook source

Report asset	Notebook source cell (section heading)	Saved filename in /tmp	Key operation
Figure 1	Beta Justification — inter-visit gap distribution	fig_inter_visit_gap_distribution.png	SQL CTE: LEAD(day) per customer → PERCENTILE_APPROX gap; matplotlib hist with β=49 line.
Figure 2	Train / Validation / Test Split — temporal window diagram	(printed plain text plus axvline overlays on Figure on monthly volume)	Python helper printing R_TRAIN/R_VAL/R_TEST ranges; overlay axvlines in fig_monthly_transaction_volume.png.
Figure 3	Train / Validation / Test Split — stability bar chart	fig_split_churn_stability.png	pandas DataFrame of len(y_*) and y_*.mean(); twin-axis matplotlib bar+line.
Figure 4	Final Test Evaluation — ROC and PR curves	fig_roc_pr_curves.png	sklearn.metrics.roc_curve + precision_recall_curve on test_prob; matplotlib subplots.
Figure 5	Final Test Evaluation — confusion matrix	fig_confusion_matrix.png	sklearn.metrics.confusion_matrix on test_pred; matplotlib imshow with annotations.
Figure 6	Threshold Analysis	fig_threshold_analysis.png	Python loop sweeping 0.05–0.95; precision/recall/F1 from sklearn; matplotlib twin plots.
Figure 7	Permutation Importance	fig_permutation_importance.png	sklearn.inspection.permutation_importance on best_clf with X_val_sc, n_repeats=10, scoring=roc_auc.
Figure 8	Churner vs Non-Churner comparison	fig_churner_comparison.png	pandas group-mean on top-10 PI features; normalised dual-bar matplotlib.
Table 1	Beta Sensitivity Analysis	(printed pandas DataFrame; no figure file)	Python loop over BETA_CANDIDATES = [18, 26, 30, 35, 40, 49]; spark.sql per β.
Table 2	Modelling — “CLEAN MODEL	(printed pandas DataFrame; no figure file)	Aggregated from results_val list + final_test_row; rendered with pandas to_string.

	COMPARISON TABLE — REPORT VERSION”		
Table 3	Pen Portraits — “PEN PORTRAIT SUMMARY TABLE — TRAINING SET”	(printed pandas DataFrame; no figure file)	pandas group-means and medians on 14 key features filtered by y_train.
Additional figures used in notebook but optional in report	Beta Sensitivity bar+line chart; Default model AUC bars; RF tuning heatmap; Feature correlation matrix; Top feature differences; Predicted risk distribution; All-models ROC-AUC.	fig_beta_sensitivity.png; fig_default_model_auc_comparison.png; fig_rf_tuning_heatmap.png; fig_feature_correlation_matrix_train.png; fig_top_feature_differences.png; fig_predicted_risk_distribution.png; fig_all_models_roc_auc.png	Each generated in the corresponding markdown section as referenced by name.

Each Databricks markdown heading visible in the notebook table of contents (Beta Justification, Label Construction, Feature Engineering, Train / Validation / Test Split, Modelling, Permutation Importance, Final Test Evaluation, Pen Portraits, Threshold Analysis) corresponds to the section heading column above. The notebook is therefore navigable directly from the report — every reported metric or figure can be located by its section title.

4. Notebook Code Commenting Policy

Comments inside the notebook describe the analytical intent of the code, not the syntax. Each user-defined function (build_feature_sql, save_features, save_labels, load_dataset, fit_temporal_group_scaler, apply_temporal_group_scaler, eval_metrics) carries a short docstring or above-block comment naming what it does, what its parameters mean and what it returns. Within longer SQL blocks, in-line comments explain the purpose of each CTE and any non-obvious choice (e.g. why visits are counted as DISTINCT day rather than as receipt counts, why strict and non-strict bounds differ at the reference date). Reading only the comments should give a high-level walkthrough of the analytical pipeline.